

Term Project – Proposal

Fit Tracker

Dozent: Stephan Bökelmann sboekelmann@ep1.rub.de Technische Hochschule Georg Agricola, Bochum

Author: Ahmad Hoteit

Module: Introduction to Database Management Systems · THGA Bochum

Submitted to: Stephan Bökelmann · sboekelmann@ep1.rub.de

Target size: about 6 pages

Every figure must be explained in the running text. A figure never stands on its own: refer to it by number and say what it shows – e.g. “Figure 1 shows the ER model of ...”.

Every figure below is `example-figure.jpg`, a dummy image shipped with this template. Photograph or scan your own sketch, drop the file next to `proposal.tex`, and point `\includegraphics` at it:

```
\includegraphics[width=0.85\linewidth]{er-sketch.jpg}
```

The `width` option scales the image to a fraction of the text width; the file extension may be omitted. Keep the `\caption` and the `\label` as they are.

1 Application domain and motivation

Fit Tracker is a self-hosted database application designed to record and manage strength training sessions. It solves the problem of tracking gym progress by replacing disorganized pen-and-paper logs and bloated, expensive commercial fitness apps. The primary users are gym-goers and strength training enthusiasts who want a fast, lightweight, and private way to log their workouts.

Usage Scenarios:

a- Logging a Session: A user finishes a set of bench presses at the gym. They open the Fit Tracker desktop client, create a new workout entry for the day, select "Bench Press" from the exercise dictionary, and log the exact weight, sets, and repetitions performed to ensure they are hitting their progressive overload targets.

b- Reviewing History: Before starting a workout, a user queries their past sessions to check what weight they lifted for squats the previous week, allowing them to know exactly how much weight to load on the bar for their current session.

2 Data model – ER sketch

Figure 1 shows the ER model for the Fit Tracker system. The central entity types are User, Workout, and Exercise. Their primary key attributes are `user_id`, `workout_id`, and `exercise_id`,

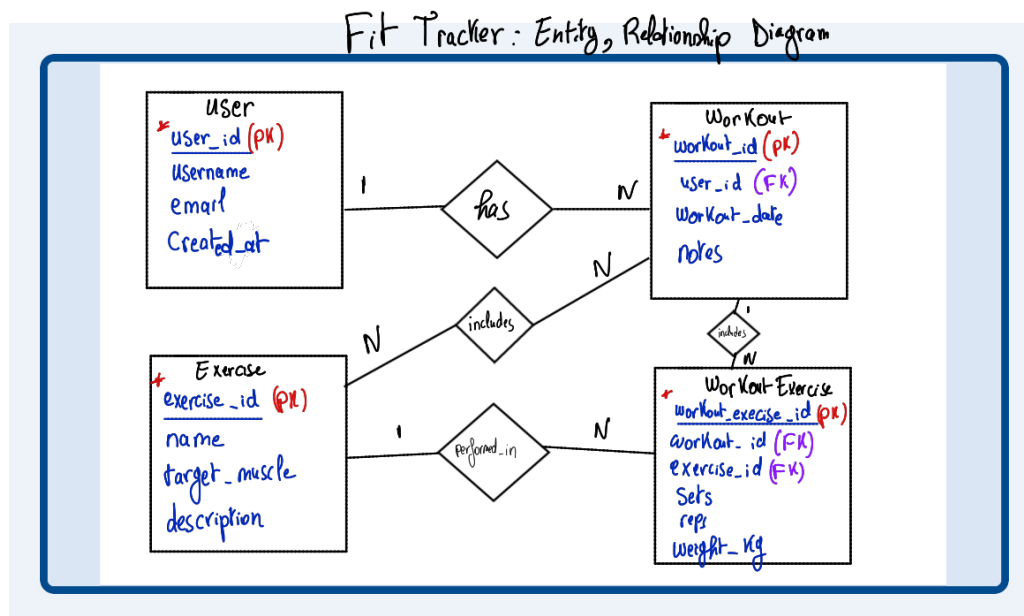


Abbildung 1: ER sketch of the Fit Tracker data model: entity types, key attributes, relationships and cardinalities (at least one N:M).

respectively. A User can have multiple Workouts (a 1:N relationship). The core of the system is the N:M relationship between Workout and Exercise: a single workout session includes multiple exercises, and a specific exercise is performed across many workouts. This N:M relationship is resolved via the WorkoutExercise entity, which captures the specific details of each performance (sets, reps, and weight).

3 From model to schema

The relational schema for the Fit Tracker system is derived as follows:

- **User**(user_id, username, email, created_at)
- **Exercise**(exercise_id, name, target_muscle, description)
- **Workout**(workout_id, *user_id*, workout_date, notes)
- **WorkoutExercise**(workout_exercise_id, *workout_id*, *exercise_id*, sets, reps, weight_kg)

Normalisation. The schema is in the Third Normal Form (3NF) because all non-key attributes depend entirely on the primary key of their respective tables. Furthermore, there are no transitive dependencies between non-key attributes, and the N:M relationship is fully resolved via the junction table to prevent redundant data.

4 API design

The planned FastAPI endpoints for the Fit Tracker system are listed below. The read operations include necessary aggregations, while all write operations are secured to prevent unauthorized modifications.

Method	Path	Purpose
GET	/users/{user_id}/workouts	Reads workout history (JOINS Workout, WorkoutExercise, and Exercise to aggregate full session details).
GET	/exercises	Reads the static dictionary of available exercises.
POST	/workouts	Creates a new workout session (requires X-API-Key).
POST	/workouts/{workout_id}/exercises	Logs specific sets, reps, and weights to an existing workout (requires X-API-Key).

5 Frontend prototype (hand sketches)

The frontend is a lightweight desktop application built using Python and Tkinter. This framework is recommended for its straightforward native GUI components and ability to be easily packaged into a .deb installer. The frontend is delivered as a .deb installer and asks for the API URL (including the specific port assigned on the lecture server) and the X-API-Key in a connection dialog at startup.

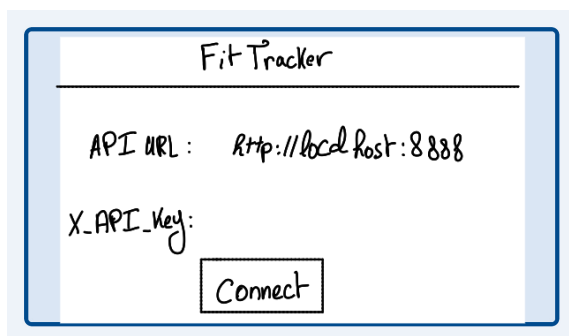


Abbildung 2: Startup connection dialog: API URL + X-API-Key.

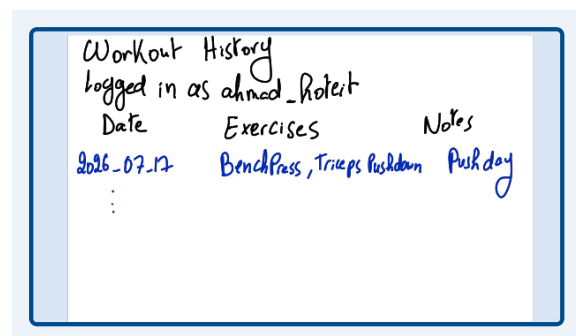


Abbildung 3: Main screen: Workout history list, calls GET /users/{user_id}/workouts.

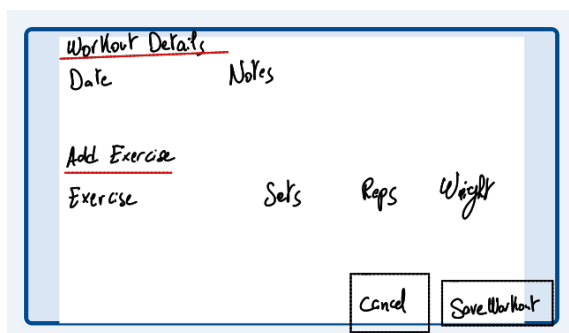


Abbildung 4: Detail form: Log new session and sets, calls POST /workouts and POST /workouts/{workout_id}/exercises.

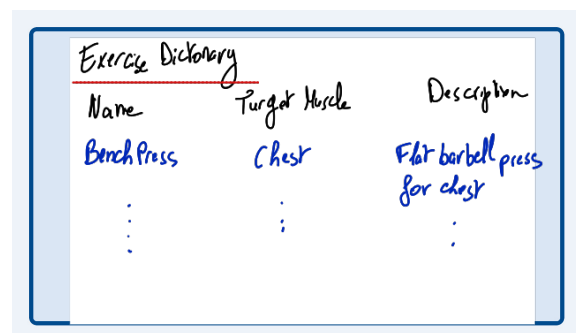


Abbildung 5: Exercise Dictionary: Lists available movements, calls GET /exercises.

Figures 2–5 show the four screens of the prototype. Figure 2 shows the connection dialog that collects the API URL and X-API-Key at startup; Figure 3 shows the main screen displaying

the user's workout history and the read endpoint it calls; Figure 4 shows the form used to log a new workout along with its specific sets and the write endpoints it calls; and Figure 5 shows the exercise dictionary screen displaying target muscles.

6 Operation with Docker Compose

The system is containerized using Docker to ensure a consistent deployment environment. The `docker-compose.yml` file orchestrates two primary services:

- **postgres:** The database service running the PostgreSQL image, which persists data via Docker volumes to ensure no workout records are lost.
- **api:** The backend service running the FastAPI application, which connects internally to the `postgres` container.

All sensitive configuration details, including the database credentials and the `X-API-Key`, are stored securely in a `.env` file. This file is explicitly excluded from version control to maintain security.

For deployment on the lecture server, the repository is cloned and the `.env` file is populated with the production credentials. The containers are then built and started in detached mode using Docker Compose. To make the backend accessible without conflicts, the internal port of the `api` container is mapped directly to the specific port assigned to this project on the lecture server.

7 Scope, milestones and risks

Scope. The Fit Tracker system is scoped to exactly 4 relational tables (User, Exercise, Workout, WorkoutExercise) and 4 FastAPI endpoints (two GET reads and two POST writes).

Milestones and schedule. The project will be executed in a single intensive development sprint following the conclusion of examinations, totaling roughly 40 hours of work.

Sprint Date	Milestone	Est. hours
29.07.2026	Schema setup, Docker configuration, and seed data ready	8
30.07.–31.07.	API read/write implementation, protected by X-API-Key	12
01.08.–02.08.	Frontend operates core endpoints, <code>.deb</code> packaging	10
03.08.–04.08.	Acceptance testing, final documentation, and demo video	10

Risks. There are two main risks that could impact the project:

- **Deployment constraints:** Port conflicts or network permission issues could arise when deploying the backend containers to the shared lecture server.
- **Packaging complexities:** Unexpected system dependencies or missing libraries might cause failures when packaging the Python/Tkinter frontend into a standalone `.deb` installer.

8 Acceptance criteria and testing

The system is considered “done and correct” when all constraints are enforced at the database level, the API accurately handles all defined requests securely, and the frontend successfully installs and connects to the backend in a production-like environment.

Acceptance criteria.

- A workout or exercise record that violates a database constraint (e.g., negative weight) is rejected (API returns 422/400).
- `GET /users/{user_id}/workouts` returns the correct aggregated session details (verifying the JOIN between Workout, WorkoutExercise, and Exercise).
- Write endpoints (`POST /workouts` and `POST /workouts/{workout_id}/exercises`) without a valid X-API-Key are refused (API returns 401/403).
- The Python/Tkinter frontend installs cleanly via the `.deb` package and successfully connects to the lecture server port.

Test plan.

- **Database:** Verify that PK/FK constraints (ensuring orphan workout exercises cannot exist) and NOT NULL constraints (e.g., `workout_date`, `target_muscle`) are strictly enforced.
- **API:** Automated endpoint tests covering both the happy path and error cases, executed with `pytest` and FastAPI's `TestClient` against a clean test database.
- **Business rule:** To address the primary risks, a clean-room installation test will verify the `.deb` package dependencies, and a deployment test will confirm the API binds successfully to the assigned lecture server port without configuration conflicts.

Effort budget: the whole term project (system, documentation and video) should fit into roughly **40 h of work** over at most **2 months**. Keep your scope within that budget.